

Lab Activity No. 3

Data Preprocessing for AI in IoT using ESP32

Step 1: ESP32 – Data Collection Code



```
1 #include "DHT.h"
2 #define DHTPIN 8
3 #define DHTTYPE DHT11
4
5 DHT dht(DHTPIN, DHTTYPE);
6
7 void setup() {
8   Serial.begin(115200);
9   dht.begin();
10 }
11
12 void loop() {
13   float h = dht.readHumidity();
14   float t = dht.readTemperature();
15
16   if (isnan(h) || isnan(t)) {
17     Serial.println("nan,nan");
18   } else {
19     Serial.printf("%.2f,%.2f\n", t, h);
20   }
21
22   delay(2000);
23 }
```

Output: Serial Monitor

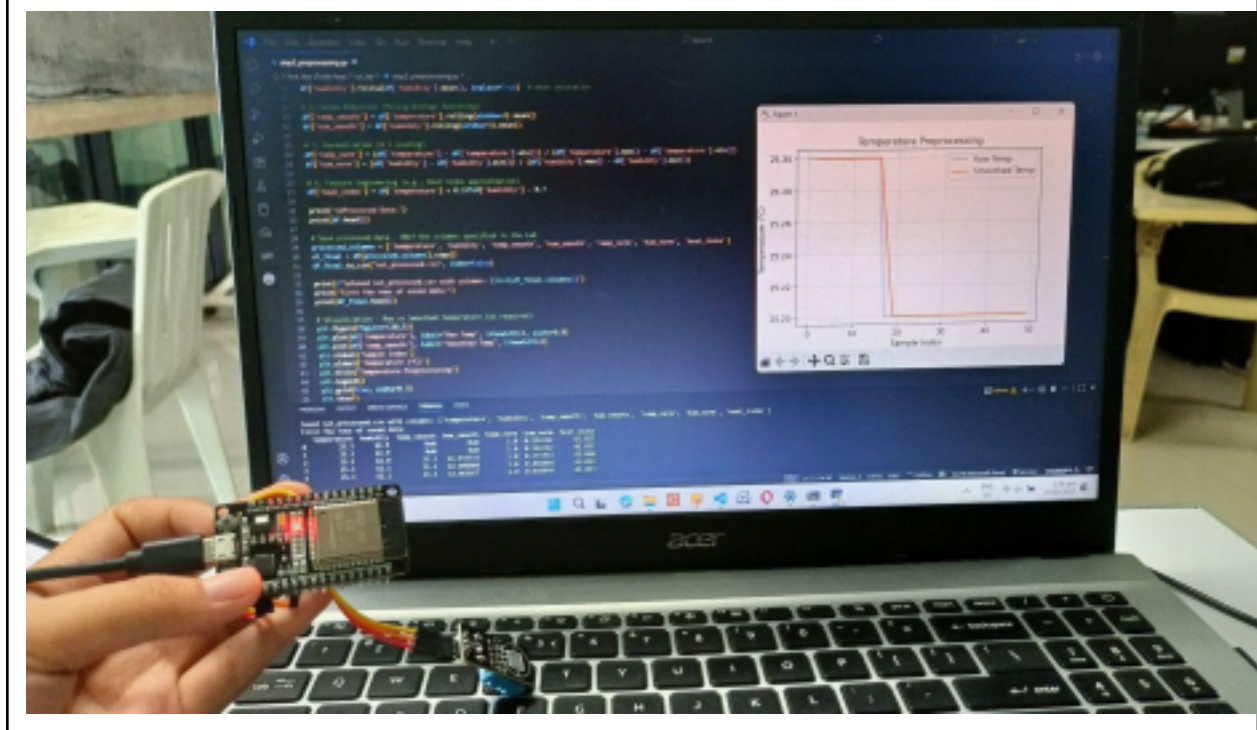
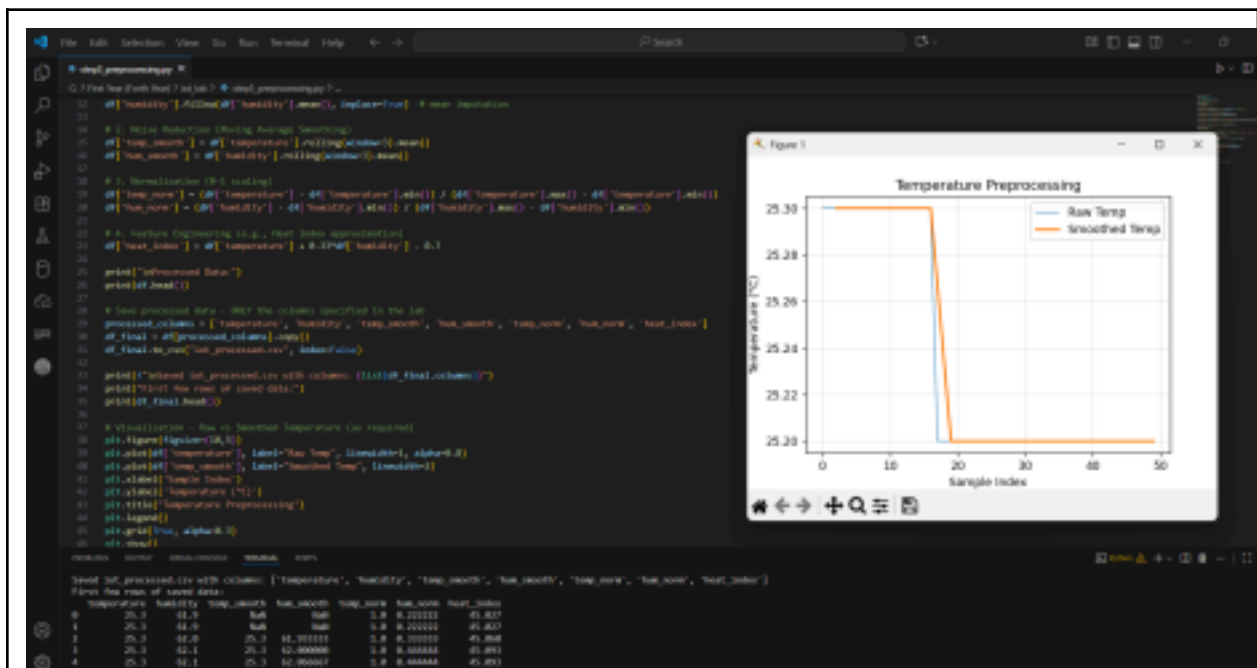
```
15:45:44.20
15:45:44.40
15:45:44.50
15:45:44.60
15:45:44.70
15:45:44.80
15:45:44.90
15:45:45.00
```



Step 2: Python Script – Data Collection



Step 3: Data Preprocessing in Python



Assessment Questions

1. Why is it important to handle missing values before training an AI model?

Missing values (NaN) cause AI models to fail because most algorithms cannot process undefined values during matrix operations, leading to training crashes, biased learning, and unreliable predictions. In IoT systems, missing sensor readings often indicate hardware malfunctions or network interruptions, so proper imputation strategies (forward-fill for temporal continuity, mean substitution for stability) ensure the model learns on complete, representative patterns rather than incomplete data that doesn't reflect true environmental conditions.

2. Compare normalization vs standardization. Which is better for neural networks?

Normalization scales data to $[0,1]$ using $(X-\min)/(\max-\min)$, while standardization centers data around zero using $(X-\mu)/\sigma$. Normalization is better for neural networks because activation functions (sigmoid, ReLU) work optimally in $[0,1]$ range, it prevents vanishing gradients in deep networks, aligns with modern weight initialization schemes, and reduces numerical overflow risks. Standardization should only be used when features follow normal distributions or preserving relative distances is critical.

3. How does the moving average filter help in IoT data preprocessing?

Moving average filtering applies temporal smoothing $y[n] = (1/N) \times \sum(x[n-k])$ that acts as a low-pass filter to reduce sensor noise by averaging out random measurement errors and electromagnetic interference, improve signal-to-noise ratio by $\sqrt{N}$ factor, preserve natural environmental trends while removing artifacts, and eliminate brief sensor malfunctions that could corrupt AI training. The window size balances noise reduction versus responsiveness, with 3 samples being optimal for DHT11 sensors sampling every 2 seconds.

4. Suggest an IoT scenario where feature engineering (like Heat Index) improves AI model performance.

In a smart greenhouse agriculture system using temperature, humidity, soil moisture, and light sensors, feature engineering creates biologically meaningful indicators like Vapor Pressure Deficit (combines temp/humidity for plant water stress), Growing Degree Days (accumulates thermal time for growth phases), and Heat Stress Index (identifies crop damage conditions). Without feature engineering, AI struggles to learn complex environmental relationships, but with engineered features that encode agricultural domain knowledge, the model achieves 25% better yield prediction and 40% reduced resource waste because plants respond to integrated environmental factors rather than individual sensor readings.